

OpenMRS QA Automation — Complete Project Guide

- [OpenMRS QA Automation — Complete Project Guide](#)
 - [Table of Contents](#)
 - [1. What Is This Project?](#)
 - [2. The Application Under Test \(O2 Demo\)](#)
 - [3. Tools & Technologies Used](#)
 - [4. Project Folder Structure](#)
 - [5. Prerequisites — What You Need to Install](#)
 - [6. How to Run the Project — Step by Step](#)
 - [7. How API Testing Works — Manual Commands](#)
 - [8. What the Automated API Tests Do](#)
 - [9. What We Fixed to Make Tests Pass](#)
 - [10. Current Rubric Status — Where We Are](#)
 - [11. What Still Needs to Be Done](#)
 - [12. Quick Reference Cheat Sheet](#)

OpenMRS QA Automation — Complete Project Guide

Course: Quality Engineer (AI & Test Automation)

Capstone: Week 6 Day 5 — Cross-Platform QA Showcase

Application Under Test: OpenMRS 2.x Reference Application

Team: Sai Ram Burri · Rohit Bharti

Date: June 2026

Table of Contents

1. [What Is This Project?](#)
2. [The Application Under Test \(O2 Demo\)](#)
3. [Tools & Technologies Used](#)
4. [Project Folder Structure](#)
5. [Prerequisites — What You Need to Install](#)
6. [How to Run the Project — Step by Step](#)
7. [How API Testing Works — Manual Commands](#)
8. [What the Automated API Tests Do](#)
9. [What We Fixed to Make Tests Pass](#)
10. [Current Rubric Status — Where We Are](#)
11. [What Still Needs to Be Done](#)
12. [Quick Reference Cheat Sheet](#)

1. What Is This Project?

This capstone project demonstrates real-world QA (Quality Assurance) automation against **OpenMRS**, an open-source Electronic Medical Record (EMR) system used by hospitals in developing countries worldwide.

We test OpenMRS in two layers:

Layer	What It Tests	Tool Used
API Layer	The backend REST API — creating patients, searching, deleting	REST Assured (Java)
UI Layer	The browser — logging in, registering patients, searching	Selenium WebDriver
BDD Layer	User stories written in plain English	Cucumber
Load Layer	How the server behaves under heavy traffic	Apache JMeter

Why two layers? Because a bug can exist in the API without being visible in the UI (or vice versa). Professional testers always validate both.

2. The Application Under Test (O2 Demo)

✅ Is the demo website working?

YES — the O2 server is fully operational.

Property	Value
Demo URL	https://o2.openmrs.org/openmrs/login.htm
API Base URL	https://o2.openmrs.org/openmrs/ws/rest/v1
Username	admin
Password	Admin123
Location	Any (select “Amani Hospital”)
OpenMRS Version	2.x Reference Application
API Documentation	https://rest.openmrs.org/#openmrs-rest-api

Note: We previously used `demo.openmrs.org` (OpenMRS 3.x), which was broken with a database migration error. We switched to `o2.openmrs.org` (OpenMRS 2.x) which is stable and fully functional.

How to Access the UI Manually

- 1. Open your browser
- 2. Go to: `https://o2.openmrs.org/openmrs/login.htm`
- 3. Enter Username: `admin`
- 4. Enter Password: `Admin123`
- 5. Select Location: `Amani Hospital`
- 6. Click **Log In**

You will see the OpenMRS home dashboard with options to register patients, find patients, schedule appointments, etc.

3. Tools & Technologies Used

Tool	Purpose	Why We Use It
Java 21	Programming language for all tests	Industry standard for enterprise testing
Maven 3.9+	Build tool — compiles and runs tests	Manages all library dependencies automatically
JUnit 5	Test framework — defines and runs individual test methods	Modern Java testing standard
REST Assured	API testing library for Java	Makes HTTP requests in human-readable syntax
Selenium WebDriver	Browser automation library	Controls Chrome/Firefox programmatically
Cucumber	BDD framework — tests written in plain English (Gherkin)	Non-technical stakeholders can read and write tests
Java Faker	Generates realistic fake data (names, dates)	Avoids test data collisions on shared servers
WebDriverManager	Auto-downloads correct ChromeDriver version	No manual driver installation needed
Apache JMeter	Load testing tool	Simulates hundreds of concurrent users

Why REST Assured?

REST Assured lets you write API tests that read almost like English:

```
given()
    .auth().preemptive().basic("admin", "Admin123")
.when()
    .get("https://o2.openmrs.org/openmrs/ws/rest/v1/session")
.then()
    .statusCode(200)
    .body("authenticated", equalTo(true));
```

Compare this to manually opening Postman or writing raw HTTP code — REST Assured is far more readable and maintainable.

4. Project Folder Structure

```
OMRS-Capstone/
|
|— docs/
|   |— Test_Plan.md           ← Component 1 (DONE)
|   |— Test_Cases.md        ← Component 2 (DONE - 20 test cases)
|   |— Project_Guide.md     ← This document
|   |— defect_reports.txt    ← Component 6 (TODO)
|   |— test_execution_summary.txt ← Component 7 (TODO)
|   |— accessibility_findings.txt ← Component 9 (TODO)
|
|— openmrs-qa-automation/    ← Maven Java project
|   |— pom.xml               ← All library dependencies declared here
|   |— src/
|       |— test/
|           |— java/
|               |— api/
|                   |— PatientApiTest.java ← Component 3 - 9 API tests (DONE ✅)
|                   |— base/
|                       |— BaseTest.java ← Shared REST Assured setup
|               |— ui/
|                   |— LoginUITest.java ← Component 4 - UI tests (TODO)
|                   |— pages/
|                       |— BasePage.java ← Page Object Model classes (TODO)
|                       |— LoginPage.java
|                       |— RegisterPatientPage.java
|                       |— SearchPatientPage.java
|                   |— bdd/steps/ ← Cucumber step definitions (TODO)
|                       |— utils/
|                           |— ApiClient.java ← REST Assured request spec factory
|                           |— ConfigLoader.java ← Reads config.properties
|                           |— DriverFactory.java ← Creates WebDriver instances
|                       |— resources/
```


Step 4 — Verify Network Access

```
curl -s -o /dev/null -w "%{http_code}" https://o2.openmrs.org/openmrs/ws/rest/v1/session -u admin:Admin123 -H "Accept: application/json"
```

Expected output: 200

If you see 200 , the O2 server is reachable and your credentials work.

6. How to Run the Project — Step by Step

Navigate to the Project Directory

```
cd /Users/sairam/Desktop/Cognizant/OMRS-Capstone/openmrs-qa-automation
```

Run API Tests Only (Component 3 — Current Focus)

```
mvn clean test -Dtest=PatientApiTest
```

What this does: 1. `mvn clean` — deletes old compiled files from `target/` folder 2. `mvn test` — compiles all test files and runs them 3. `-Dtest=PatientApiTest` — runs ONLY the PatientApiTest class

Expected output:

```
[INFO] Running api.PatientApiTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Run All Tests

```
mvn clean test
```

Run UI Tests Only (once implemented)

```
# Headless mode (no visible browser - faster, good for CI)
mvn test -Dtest=LoginUITest -Dheadless=true
```

```
# Visible browser (good for debugging and demos)
mvn test -Dtest=LoginUITest -Dheadless=false
```

Run BDD/Cucumber Tests (once implemented)

```
mvn test -Dtest=RunCucumberTest
```

View Test Reports

After running, Surefire generates reports in:

```
openmrs-qa-automation/target/surefire-reports/
```

Open the `.txt` file to see which tests passed/failed and why.

7. How API Testing Works — Manual Commands

Before writing automated tests, QA engineers first test APIs **manually** using tools like `curl` (command line) or Postman (GUI). Here is exactly how to test the OpenMRS API manually.

Understanding HTTP Methods

HTTP Method	Action	Example
GET	Retrieve data	Get a patient record
POST	Create data	Create a new patient
PUT	Update data	Update patient info
DELETE	Remove/void data	Void (soft-delete) a patient

Authentication

OpenMRS uses **HTTP Basic Authentication** — you send username:password with every request, encoded in Base64.

```
# -u admin:Admin123 is the shorthand for Basic Auth in curl
curl -u admin:Admin123 https://o2.openmrs.org/openmrs/ws/rest/v1/session
```

TC-API-001: Test Valid Login (GET /session)

What it tests: Does the API accept correct credentials and return `authenticated: true` ?

```
curl -s \  
  -u admin:Admin123 \  
  -H "Accept: application/json" \  
  https://o2.openmrs.org/openmrs/ws/rest/v1/session | python3 -m json.tool
```

Expected response:

```
{  
  "authenticated": true,  
  "user": {  
    "username": "admin",  
    "display": "admin"  
  }  
}
```

TC-API-002: Test Invalid Login (GET /session)

What it tests: Does the API reject wrong credentials with `authenticated: false` ?

```
curl -s \  
  -u wrong:wrongpassword \  
  -H "Accept: application/json" \  
  https://o2.openmrs.org/openmrs/ws/rest/v1/session | python3 -m json.tool
```

Expected response:

```
{  
  "authenticated": false  
}
```

Note: OpenMRS returns HTTP 200 with `authenticated: false` instead of HTTP 401. This is intentional design.

TC-API-003: Create a New Patient (POST /patient)

What it tests: Can we create a patient via the API?

Important: OpenMRS 2.x requires: - Patient data wrapped inside a `"person"` key - At least one valid `"identifiers"` entry with a Luhn Mod-30 check digit ID

First, generate a valid patient ID (or pick one from the list below):

Valid IDs for testing

10001A

10002P

10003H

10004W

10005F

```
curl -s -X POST \
  -u admin:Admin123 \
  -H "Content-Type: application/json" \
  -H "Accept: application/json" \
  -d '{
    "person": {
      "names": [{ "givenName": "John", "familyName": "TestPatient" }],
      "gender": "M",
      "birthdate": "1990-05-15"
    },
    "identifiers": [{
      "identifier": "10001A",
      "identifierType": "05a29f94-c0ed-11e2-94be-8c13b969e334",
      "location": "aff27d58-a15c-49a6-9beb-d30dcfc0c66e",
      "preferred": true
    }]
  }' \
  https://o2.openmrs.org/openmrs/ws/rest/v1/patient | python3 -m json.tool
```

Expected response (HTTP 201 Created):

```
{
  "uuid": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "identifiers": [...],
  "person": {
    "display": "TestPatient, John"
  }
}
```

Save the **UUID** from the response — you need it for the next tests.

TC-API-004: Fetch a Patient by UUID (GET /patient/{uuid})

Replace `{UUID}` with the UUID you got from the create step:

```
curl -s \  
  -u admin:Admin123 \  
  -H "Accept: application/json" \  
  https://o2.openmrs.org/openmrs/ws/rest/v1/patient/{UUID} | python3 -m json.tool
```

Expected response (HTTP 200):

```
{  
  "uuid": "{UUID}",  
  "voided": false,  
  "identifiers": [...],  
  "person": {  
    "display": "TestPatient, John"  
  }  
}
```

TC-API-005: Search for a Patient by Name (GET /patient?q=NAME)

```
curl -s \  
  -u admin:Admin123 \  
  -H "Accept: application/json" \  
  "https://o2.openmrs.org/openmrs/ws/rest/v1/patient?q=TestPatient" | python3 -m json.tool
```

Expected response (HTTP 200):

```
{  
  "results": [  
    {  
      "uuid": "{UUID}",  
      "display": "10001A - TestPatient, John"  
    }  
  ]  
}
```

TC-API-006: Delete (Void) a Patient (DELETE /patient/{uuid})

```
curl -s -X DELETE \
-u admin:Admin123 \
-H "Accept: application/json" \
-o /dev/null -w "HTTP Status: %{http_code}\n" \
"https://o2.openmrs.org/openmrs/ws/rest/v1/patient/{UUID}?reason=manual+test+cleanup"
```

Expected response: HTTP Status: 204

Then verify the patient is now voided:

```
curl -s \
-u admin:Admin123 \
-H "Accept: application/json" \
https://o2.openmrs.org/openmrs/ws/rest/v1/patient/{UUID} | python3 -c "import json,sys;
d=json.load(sys.stdin); print('voided:', d['voided'])"
```

Expected: voided: True

TC-API-007: Test 404 for Non-Existent UUID

```
curl -s -o /dev/null -w "HTTP Status: %{http_code}\n" \
-u admin:Admin123 \
-H "Accept: application/json" \
https://o2.openmrs.org/openmrs/ws/rest/v1/patient/00000000-0000-0000-0000-000000000000
```

Expected: HTTP Status: 404

TC-API-008: Test 400 for Bad Request (POST /patient)

```
curl -s -o /dev/null -w "HTTP Status: %{http_code}\n" \
-X POST \
-u admin:Admin123 \
-H "Content-Type: application/json" \
-H "Accept: application/json" \
-d '{}' \
https://o2.openmrs.org/openmrs/ws/rest/v1/patient
```

Expected: HTTP Status: 400

What Tool to Use for API Testing

Tool	Best For	How to Access
curl (command line)	Quick one-off tests, scripting	Already installed on Mac/Linux
Postman (GUI)	Visual testing, saving collections	Download from postman.com
REST Assured (Java)	Automated regression tests in code	Already in our pom.xml
Browser Dev Tools	Watching API calls the UI makes	F12 → Network tab in any browser

For this project we use **REST Assured** because it integrates directly into our Maven build and runs automatically with `mvn test`.

8. What the Automated API Tests Do

The file `PatientApiTest.java` contains 9 automated tests that run in order. Here is what each one does and why:

Test #	Method Name	What It Does	Why It Matters
TC-001	<code>validCredentials_sessionReturnsAuthenticatedTrue</code>	Logs in with correct credentials, checks <code>authenticated:true</code>	Baseline — if auth doesn't work, nothing else will
TC-002	<code>invalidCredentials_sessionReturnsAuthenticatedFalse</code>	Tries wrong password, checks <code>authenticated:false</code>	Security — the API must reject bad credentials
TC-003	<code>createPatient_returns201WithNonNullUuid</code>	Creates a new patient, saves the UUID for later tests	Core functionality — patient creation
TC-004	<code>fetchPatientByUuid_returns200WithMatchingUuidAndNotVoided</code>	Fetches the patient created in TC-003	Verifies the patient was actually saved
TC-005	<code>searchByGivenName_resultsNonEmptyAndContainsCreatedUuid</code>	Searches by name, verifies our patient appears	Search is a critical

Test #	Method Name	What It Does	Why It Matters
			workflow for nurses
TC-006	<code>deletePatient_returns200AndSubsequentGetShowsVoidedTrue</code>	Deletes (soft-deletes) the patient, verifies voided flag	Soft-delete is safer than hard-delete in EMR
TC-007	<code>fetchNonExistentPatient_returns404</code>	GETs a fake UUID, expects 404 Not Found	Edge case — system handles bad input gracefully
TC-008	<code>createPatientEmptyBody_returns400</code>	POSTs <code>{}</code> , expects 400 Bad Request	Input validation — server rejects garbage input
TC-009	<code>chainedLifecycle_createFetchDeleteVerifyVoided</code>	Full end-to-end: create → fetch → delete → verify voided	Integration test covering the whole patient lifecycle

Test Dependencies

TC-004, TC-005, and TC-006 depend on TC-003 creating a patient first (they share a UUID). If TC-003 fails, those three tests automatically **skip** (not fail) using `assumeTrue()`. This is intentional — no point testing “fetch patient” if “create patient” broke.

9. What We Fixed to Make Tests Pass

Here is a chronological log of every bug found and fixed during this session:

Problem 1: Wrong Server — demo.openmrs.org Was Broken

Symptom: All 9 tests failing with HTML response instead of JSON.

Root Cause: `demo.openmrs.org` (OpenMRS 3.x) had a database migration error:

```
Table 'openmrs-db.relationship' doesn't exist
```

The server was returning an error HTML page for every request.

Fix: Changed `config.properties` from:

```
base.url=https://demo.openmrs.org/openmrs/ws/rest/v1
```

to:

```
base.url=https://o2.openmrs.org/openmrs/ws/rest/v1
```

Problem 2: sessionId Field Does Not Exist in OpenMRS 2.x

Symptom: TC-API-001 failing — `JSON path sessionId doesn't match. Expected: not null, Actual: null`

Root Cause: OpenMRS 3.x returned a `sessionId` field in the `/session` response. OpenMRS 2.x does not.

Fix: Changed the assertion from checking `sessionId` to checking `user` (which IS returned):

```
// Before (broken):  
.body("sessionId", notNullValue());  
  
// After (fixed):  
.body("user", notNullValue());
```

Problem 3: Patient Payload Format Changed in OpenMRS 2.x

Symptom: TC-003 failing — `Expected status code <201> but was <400>`. Error: `"The person property is missing"`

Root Cause: OpenMRS 3.x accepted flat patient JSON:

```
{ "names": [...], "gender": "M", "birthdate": "1990-01-01" }
```

OpenMRS 2.x requires patient demographics to be nested under a `"person"` key.

Fix: Updated `patientJson()` to use the 2.x format:

```
{
  "person": {
    "names": [{ "givenName": "John", "familyName": "Doe" }],
    "gender": "M",
    "birthdate": "1990-01-01"
  },
  "identifiers": [{
    "identifier": "12345X",
    "identifierType": "05a29f94-c0ed-11e2-94be-8c13b969e334",
    "location": "aff27d58-a15c-49a6-9beb-d30dcfc0c66e",
    "preferred": true
  }]
}
```

Problem 4: Patient Identifiers Require Luhn Mod-30 Check Digit

Symptom: After fixing the payload format — "Identifier TEST-001 is not appropriate for validation scheme Luhn Mod-30 Check-Digit Validator"

Root Cause: OpenMRS uses a custom check digit algorithm (Luhn adapted for base-30, 30-character alphabet 0123456789ACDEFGHJKLMNPRTUVWXY) to validate patient IDs and prevent typos.

How Luhn Mod-30 Works: 1. Start from the rightmost character (this one IS doubled) 2. Alternate: double, not double, double, not double... 3. When a doubled value ≥ 30 : reduce using $\text{value}/30 + \text{value}\%30$ (NOT simple subtraction) 4. Sum all values 5. Check digit = $\text{chars}[(30 - \text{sum}\%30) \% 30]$

Fix: Implemented the algorithm as a helper method in Java:

```
private static String generatePatientId() {
    int base = 10000 + RNG.nextInt(89999);
    String baseStr = String.valueOf(base);
    int sum = 0;
    boolean doubled = true; // rightmost IS doubled
    for (int i = baseStr.length() - 1; i >= 0; i--) {
        int val = LUHN_CHARS.indexOf(baseStr.charAt(i));
        if (doubled) {
            val *= 2;
            if (val >= LUHN_CHARS.length()) {
                val = val / LUHN_CHARS.length() + val % LUHN_CHARS.length();
            }
        }
        sum += val;
        doubled = !doubled;
    }
    int checkIdx = (LUHN_CHARS.length() - (sum % LUHN_CHARS.length())) % LUHN_CHARS.length();
    return baseStr + LUHN_CHARS.charAt(checkIdx);
}
```

Problem 5: DELETE Returns 204, Not 200

Symptom: TC-006 failing — Expected status code <200> but was <204>

Root Cause: OpenMRS 2.x REST API returns **204 No Content** for successful soft-deletes. HTTP 204 means “success, but there is no body to return.”

Fix: Changed both `deletePatient()` helper and TC-006 assertion:

```
// Before:
.statusCode(200);

// After:
.statusCode(204);
```

Final Result After All Fixes

Tests run: 9, Failures: 0, Errors: 0, Skipped: 0 — BUILD SUCCESS

10. Current Rubric Status — Where We Are

#	Component	Points	Status	Notes
1	Test Plan	10	✓ DONE	docs/Test_Plan.md
2	Test Cases (min 15)	15	✓ DONE	docs/Test_Cases.md — 20 cases
3	Automated API Test Suite (min 8)	15	✓ DONE	9/9 tests pass
4	Automated UI Test Suite (min 5)	15	■ TODO	Selenium — LoginUITest.java
5	BDD Feature File (Cucumber)	10	■ TODO	patient_management.feature
6	Defect Reports (min 3)	10	■ TODO	docs/defect_reports.txt
7	Test Execution Summary	10	■ TODO	docs/test_execution_summary.txt
8	Load Test Results (JMeter)	5	■ TODO	load-test/OpenMRS_LoadTest.jmx

#	Component	Points	Status	Notes
9	Accessibility Testing Evidence	5	 TODO	<code>docs/accessibility_findings.txt</code>
10	Presentation Slides	5	 TODO	5–8 slides
Total Earned		40/100		

11. What Still Needs to Be Done

Component 4 — UI Tests (Selenium, 15 pts)

Write at least 5 browser automation tests in `LoginUITest.java`:

1. Login with valid credentials → lands on home page
2. Login with invalid credentials → error message shown
3. Register a new patient via the UI form
4. Search for a patient by name
5. Log out successfully

Command to run when done:

```
mvn test -Dtest=LoginUITest -Dheadless=false
```

Component 5 — BDD Feature File (Cucumber, 10 pts)

Write 3+ scenarios in `patient_management.feature` using Gherkin syntax:

```
Feature: Patient Management

Scenario: Create a new patient via the API
  Given the admin user is authenticated
  When a POST request is sent with valid patient data
  Then the response status should be 201
  And the response should contain a UUID
```

Command to run when done:

```
mvn test -Dtest=RunCucumberTest
```

Component 6 — Defect Reports (10 pts)

Write at least 3 defect reports in `docs/defect_reports.txt`. Use the bugs we found during this project:

Defect 1: demo.openmrs.org returns HTML instead of JSON (server outage) **Defect 2:** OpenMRS 2.x does not return `sessionId` in `/session` response (API behavior change) **Defect 3:** OpenMRS 2.x DELETE patient returns 204, not 200 (REST standard change)

Each report should include: ID, Title, Severity, Steps to Reproduce, Expected Result, Actual Result, Status.

Component 7 — Test Execution Summary (10 pts)

Fill in `docs/test_execution_summary.txt` with real numbers from your test runs:

- Total test cases planned: 9 (API) + 5 (UI) + 3 (BDD) = 17
 - Tests executed: 9
 - Tests passed: 9
 - Tests failed: 0 (after all fixes)
 - Pass rate: 100%
 - Execution time: ~5 seconds (API suite)
-

Component 8 — JMeter Load Test (5 pts)

1. Download Apache JMeter from <https://jmeter.apache.org/>
 2. Open `load-test/OpenMRS_LoadTest.jmx`
 3. Run against `https://o2.openmrs.org/openmrs/ws/rest/v1/session`
 4. Target: 50 concurrent users, 10 requests each
 5. Export HTML report as screenshot
-

Component 9 — Accessibility Testing (5 pts)

Manually test the O2 UI with keyboard only:

1. Open <https://o2.openmrs.org/openmrs/login.htm>
2. Tab to username field (do NOT click)
3. Type `admin`, Tab to password, type `Admin123`
4. Tab to Location dropdown, use arrow keys to select Amani Hospital
5. Tab to Log In button, press Enter
6. Document what you found in `docs/accessibility_findings.txt`

Component 10 — Presentation Slides (5 pts)

Create 5–8 slides covering:

1. Project overview — what is OpenMRS, what are we testing
2. Test plan summary — scope, approach
3. API test results — 9/9 pass screenshot
4. UI test demo — screenshot
5. Defects found — table with severity
6. Load test results — JMeter graph
7. Lessons learned

12. Quick Reference Cheat Sheet

Key Commands

```
# Navigate to project
cd /Users/sairam/Desktop/Cognizant/OMRS-Capstone/openmrs-qa-automation

# Run API tests (the ones that currently work)
mvn clean test -Dtest=PatientApiTest

# Run all tests
mvn clean test

# Check if O2 server is up
curl -s -o /dev/null -w "%{http_code}" https://o2.openmrs.org/openmrs/ws/rest/v1/session -u
admin:Admin123

# View test results
cat target/surefire-reports/api.PatientApiTest.txt
```

Key URLs

Resource	URL
O2 Login	https://o2.openmrs.org/openmrs/login.htm
O2 REST API	https://o2.openmrs.org/openmrs/ws/rest/v1
REST API Docs	https://rest.openmrs.org/#openmrs-rest-api
OpenMRS Wiki	https://wiki.openmrs.org

Key Config Values (config.properties)

Key	Value
base.url	https://o2.openmrs.org/openmrs/ws/rest/v1
admin.username	admin
admin.password	Admin123
identifier.type.uuid	05a29f94-c0ed-11e2-94be-8c13b969e334
location.uuid	aff27d58-a15c-49a6-9beb-d30dcfc0c66e

Test Results History

Date	Server	Tests Run	Pass	Fail	Skip
Jun 29, 2026	demo.openmrs.org	9	0	6	3
Jun 29, 2026	o2.openmrs.org (initial)	9	3	3	3
Jun 29, 2026	o2.openmrs.org (fix Luhn)	9	7	2	0
Jun 29, 2026	o2.openmrs.org (fix 204)	9	9	0	0

Document generated: June 29, 2026
Project: OpenMRS QA Automation Capstone
Team: Sai Ram Burri · Rohit Bharti